GOOGLE | 1POINT3ACRES | TOPIC 5 OF 10 | CORE TOPIC

Trees and Tries

10 questions to study here, each on its own page, with the full interview answer and code

2 more fit this topic but are covered in another topic: they are listed as SKIP at the end of the contents

9	1
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Trees and Tries | 10 questions | tap a title to jump to it

TREES (7)

M 1. Expression Tree Single-Leaf Mutation **2026-09-27**

M 2. Compute File System Total Size **2026-09-02**

M 3. Nested List Weight Sum (LC 339) with Follow-Ups **2026-08-16**

M 4. Longest Consecutive Path in Binary Tree **2026-07-02**

M 5. Employee Hierarchy Add / Delete **2026-06-10**

M 6. Streaming Insert + Find K-th Largest **2026-02-04**

H 7. Huffman Tree Construction **2026-06-01**

TRIES (3)

M 8. URL Request Handler Router **2026-10-16**

M 9. Trie Prefix Search / Autocomplete **2026-07-02**

M 10. Greedy Longest-Match Dictionary Tokenizer **2026-06-15**

SKIP: these fit this topic but have their pages in another topic (2)

M SKIP Minimum-Cost Tree Disconnection **topic 2: Dynamic Programming**

H SKIP Tree Distance Sum (Re-root DP) **topic 2: Dynamic Programming**

STUDY HERE: topic 5, Trees and Tries | main topic: Trees

Also fits: Graphs and Search, Grids and Simulation (listed there as SKIP, no pages). Site tags: tree, dfs, simulation, medium, state-machine

MEDIUM | 1Point3Acres | onsite-coding | 1 of 10

Expression Tree Single-Leaf Mutation

ASKED 2026-09-27 | 23 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 203/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-27 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/expression-tree-single-leaf-mutation

QUESTION

Evaluate a boolean expression tree, then answer the root value after each single-leaf flip. Current variants include XOR/OR/AND/NOT operators, no parent pointer on the node interface, and an expectation that the candidate cache parent links or paths during the initial traversal.SWEtreedfssimulationmediumstate-machineFrequencyLowLast asked2026-09-27Stageonsite-coding

APPROACH

Practice tree evaluation for boolean expression trees.
Add parent pointers or store root paths during leaf collection to support path-only recomputation.
Drill AND / OR / XOR truth tables so updates are mechanical.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which operators exist (AND, OR, XOR, NOT), whether XOR and others are binary only, whether nodes have parent pointers, and whether leaves are flipped one at a time and then restored or cumulatively.

BRUTE FORCE TO SAY OUT LOUD

For each leaf flip it, evaluate the whole tree, and flip it back. O(n^2).

OPTIMAL IDEA

Evaluate once and cache each node's value. During that traversal record each leaf's path to the root (or parent links). To flip a leaf, recompute only the nodes on its path upward, stopping early when a parent's value does not change because the root value is then the same as before. That is O(depth) per flip.

DRY-RUN THESE CASES

A single leaf tree, a NOT at the root, XOR chains where every flip changes the root, OR with a true sibling (early stop).

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Several flips applied together, or an undo.

PITFALLS

Mutating the cached values without restoring them between independent flips.

HOW TO THINK ABOUT IT

Flipping one leaf can only change the nodes on its path to the root, so cache every node's value and recompute only that path. The path is found through parent links you build during the first traversal. Stop early when a recomputed node keeps its old value, because nothing above it can change.

Basis: Written; incremental evaluation on a tree. Assumed rules (confirm them): leaves hold booleans; internal nodes are AND, OR, XOR (binary) or NOT (unary); report the root value after flipping each leaf on its own (flip, read the root, restore).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(root, leaves):
    answers = []
    for leaf in leaves:
        leaf.value = not leaf.value
        answers.append(evaluate(root))
        leaf.value = not leaf.value  # flip back
    return answers
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

L leaves times an O(n) evaluation: O(n^2). Caching every node's value and recomputing only the path to the root (stopping when a value does not change) costs O(depth) per leaf.

OPTIMAL SOLUTION · Cache values and parents, recompute only the path with early stop

TIME	SPACE
O(n + L * depth)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The node interface has no parent pointer.

Build the path while collecting leaves: carry the list of ancestors with each stack entry, and use that list for the walk up. It costs O(depth) memory per leaf; the parent dict does the same job with O(n) total.

FOLLOW-UP 2 Apply several flips at once, or flips that stay.

The same cached values work: toggle, walk up with early stop, and keep the changes. For many flips between queries, recompute each affected path in turn.

FOLLOW-UP 3 Which leaves change the answer at all?

Compute top-down 'sensitivity': a leaf matters if every ancestor's other input lets it through (AND with true, OR with false, any XOR or NOT). One pass gives all of them in O(n).

FOLLOW-UP 4 Which leaves can change the answer at all?

Propagate a 'matters' flag downward: through AND the other side must be true, through OR the other side must be false, through XOR and NOT it always passes. A leaf matters exactly when the flag reaches it (checked against flipping every leaf). Subtree values are cached in practice, giving O(n).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Expression Tree Single-Leaf Mutation
#
# Evaluate a boolean expression tree, then answer the root
# value after each single-leaf flip. Current variants
# include XOR/OR/AND/NOT operators, no parent pointer on
# the node interface, and an expectation that the
# candidate cache parent links or paths during the initial
# traversal.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The node interface has no parent pointer.
# 2. Apply several flips at once, or flips that stay.
# 3. Which leaves change the answer at all?
# 4. Which leaves can change the answer at all?
#
# INTERVIEW NOTES
# Flipping one leaf can only change the nodes on its path
# to the root, so cache every node's value and recompute
# only that path. The path is found through parent links
# you build during the first traversal. Stop early when a
# recomputed node keeps its old value, because nothing
# above it can change.
#
# BRUTE FORCE (say it first): Flip, re-evaluate the whole
# tree, flip back
# For each of the L leaves, toggle it, evaluate the entire
# tree, and toggle it back. It is the correct baseline and
# the oracle for the cached version. Time O(n^2). Space
# O(h).


# HELPERS for the brute force
def _apply(op, a, b=None):
    return {
        "AND": lambda: a and b,
        "OR": lambda: a or b,
        "XOR": lambda: a != b,
        "NOT": lambda: not a,
    }[op]()
```

```
def _eval_full(node):
    if node.op is None:
        return node.val
    if node.op == "NOT":
        return not _eval_full(node.left)
    return _apply(
        node.op,
        _eval_full(node.left),
        _eval_full(node.right),
    )


def _leaves(root):
    out, stack = [], [root]
    while stack:
        n = stack.pop()
        if n.op is None:
            out.append(n)
        else:
            if n.right:
                stack.append(n.right)
            stack.append(n.left)
    return out


# BRUTE FORCE (say it first): Flip, re-evaluate the whole
# tree, flip back | time O(n^2)
class XNode:
    def __init__(
        self, op=None, left=None, right=None, val=None
    ):
        self.op, self.left, self.right, self.val = (
            op,
            left,
            right,
            val,
        )


def flips_brute(root):
    out = []
    for leaf in _leaves(root):
        leaf.val = not leaf.val
        out.append(_eval_full(root))
        leaf.val = not leaf.val
    return out


# HELPERS for the base question
```

```
def combine(op, a, b=None):
    # value of one gate from its input values
    if op == "AND":
        return a and b
    if op == "OR":
        return a or b
    if op == "XOR":
        return a != b
    # NOT has one input
    return not a


def right_value(value, node):
    # NOT has no right child
    return value[id(node.right)] if node.right else None


def build_cache(root):
    # one traversal caches parents, values and leaves
    parent, value, leaves = {id(root): None}, {}, []
```

```
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                parent[id(child)] = node
                stack.append(child)
    # reversed order visits children before parents
    for node in reversed(order):
        if node.op is None:
            value[id(node)] = node.val
            leaves.append(node)
        else:
            a = value[id(node.left)]
            b = right_value(value, node)
            value[id(node)] = combine(node.op, a, b)
    return parent, value, leaves


# BASE QUESTION (optimal): Cache values and parents,
# recompute only the path with early stop
# Idea: One traversal stores every node's value and its
# parent.
def flip_answers(root):
```

```
    parent, value, leaves = build_cache(root)
    answers = []
    for leaf in leaves:
        undo, node = [], leaf
        new = not value[id(leaf)]
        # walk up only while a value really changes
        while node is not None and value[id(node)] != new:
            undo.append((node, value[id(node)]))
            value[id(node)] = new
            node = parent[id(node)]
            if node is None:
                break
            a = value[id(node.left)]
            b = right_value(value, node)
            new = combine(node.op, a, b)
        answers.append(value[id(root)])
        # put every changed value back for the next leaf
        for node, old in reversed(undo):
            value[id(node)] = old
    return leaves, answers


# FOLLOW-UP 3: Which leaves change the answer at all?
# Answer: Compute top-down 'sensitivity': a leaf matters
# if every ancestor's other input lets it through (AND
# with true, OR with false, any XOR or NOT). One pass
# gives all of them in O(n).
# FOLLOW-UP 4: Which leaves can change the answer at all?
# Answer: Propagate a 'matters' flag downward: through AND
# the other side must be true, through OR the other side
# must be false, through XOR and NOT it always passes.
def sensitive_leaves(root):
    # leaves whose flip can change the root value
    _, value, _ = build_cache(root)
    found, stack = [], [(root, True)]
    while stack:
        node, matters = stack.pop()
        if node.op is None:
            if matters:
                found.append(node)
            continue
        l, r = node.left, node.right
        # the flag passes if the other side lets it
        left_ok = right_ok = matters
        if node.op == "AND":
```

```
            left_ok = matters and value[id(r)]
            right_ok = matters and value[id(l)]
        elif node.op == "OR":
            left_ok = matters and not value[id(r)]
            right_ok = matters and not value[id(l)]
        stack.append((l, left_ok))
        if r is not None:
            stack.append((r, right_ok))
    return found


# HELPERS for the checks below
class Node:
    def __init__(self, op=None, left=None,
                 right=None, val=None):
        # op is AND, OR, XOR, NOT, or None for a leaf
        self.op, self.val = op, val
        self.left, self.right = left, right


def evaluate(node):
    # plain recursive evaluation (also the checker)
    if node.op is None:
        return node.val
    a = evaluate(node.left)
    b = evaluate(node.right) if node.right else None
    return combine(node.op, a, b)


if __name__ == "__main__":
    t, f = Node(val=True), Node(val=False)
    tree = Node("XOR", Node("AND", t, Node(val=True)),
                Node("NOT", f))
    leaves, answers = flip_answers(tree)
    # every cached answer matches a full re-evaluation
    for leaf, ans in zip(leaves, answers):
        leaf.val = not leaf.val
        assert evaluate(tree) == ans
        leaf.val = not leaf.val
    # the tree is unchanged afterwards
    assert evaluate(tree) is False
    # sensitivity agrees with really flipping each leaf
    base = evaluate(tree)
    for leaf in leaves:
        leaf.val = not leaf.val
```

```
        changes = evaluate(tree) != base
        leaf.val = not leaf.val
        assert changes == (leaf in sensitive_leaves(tree))
```

NOTES FROM CANDIDATES

Brute force is: collect leaves by inorder traversal, flip each leaf, and re-evaluate the whole tree.

The natural optimization is to cache subtree values and only recompute the path from the flipped leaf to the root.

Clarify whether XOR is binary-only and whether the tree is guaranteed valid.

Early-stop optimization is expected: after a leaf flip, recompute upward only until a parent value stays unchanged, then reuse the previous root value.

Settle how intermediate update state will be represented before implementation. In an L4 onsite, uncertainty about that representation caused a promising approach to fall back to brute-force code and contributed to negative coding feedback.

PREPARATION

Practice tree evaluation for boolean expression trees.
Add parent pointers or store root paths during leaf collection to support path-only recomputation.
Drill AND / OR / XOR truth tables so updates are mechanical.

STUDY HERE: topic 5, Trees and Tries | main topic: Trees

Also fits: Graphs and Search (listed there as SKIP, no pages). Site tags: tree, dfs, medium

MEDIUM | 1Point3Acres | onsite-coding, phone-screen | 2 of 10

Compute File System Total Size

ASKED 2026-09-02 | 48 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 204/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/file-system-total-size

QUESTION

EXAMPLES

```
root = Dir(
  Dir(File(10), File(20)),
  File(5),
  Dir(File(30)),
)
Total: 65
```

APPROACH

Drill recursive-and-iterative DFS over directory-style trees.
Have ready: cycle-safe DFS (visited set), memoized DFS (cache on internal nodes), invalidation cascade.
Practice the same problem under the the canonical "longest absolute file path" parsing problem for warm-up.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the structure can contain cycles or shared directories (links), whether files have sizes only, and whether to handle very deep trees.

BRUTE FORCE TO SAY OUT LOUD

Recursive DFS adding file sizes. Simple and usually the expected first answer.

OPTIMAL IDEA

DFS with a visited set if cycles or shared nodes are possible, so each node is counted once. Use an explicit stack for deep trees. If queried repeatedly, memoize directory totals and invalidate ancestors when a file changes. O(n) per computation.

DRY-RUN THESE CASES

The example giving 65, an empty directory, a directory containing only directories, a symlink loop.

LIKELY FOLLOW-UPS

Cache totals, handle updates, or find the largest directory.

PITFALLS

INTERVIEW-DAY PLAN

Recursion depth on a long chain. Double counting shared nodes.

HOW TO THINK ABOUT IT

A total over a tree is a post-order sum. Two things change the answer in real file systems: depth (recursion limit) and sharing or cycles (the same node reachable twice must be counted once). A visited set solves both the cycle risk and the double counting, and an explicit stack solves the depth.

Basis: Written; a tree sum (like LeetCode 690 / 1448 style traversals). Assumed model: File(size) and Dir(children...); directories may be shared (links) in the harder version.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def size(node):
    if is_file(node):
        return node.size
    total = 0
    for child in node.children:
        total += size(child)
    return total
```

TIME	SPACE
O(n)	O(depth) call stack

WHY IT IS SLOW

Linear time, which is optimal; the weaknesses are the recursion depth on very deep trees and an endless loop if a link creates a cycle. An explicit stack with a visited set fixes both.

OPTIMAL SOLUTION · Iterative traversal with a visited set

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is queried often and files change.

Cache each directory's total and, when a file changes, clear the cache only for its ancestors (walk parent links). A query recomputes only cleared nodes. Updates cost O(depth).

FOLLOW-UP 2 Hard links: the same file appears under two directories.

Count a file once per identity (a visited set of ids, as coded) if the question asks for disk usage, but once per path if it asks for the sum of the sizes shown. Ask which.

FOLLOW-UP 3 Find the largest directory.

Compute totals in post-order and keep the maximum; it is the same pass. For top-K directories keep a heap of size K. **Cost:** one post-order pass, O(n); keeping the top K directories with a heap of size K makes it O(n log K).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute File System Total Size
#
# Examples: root = Dir( Dir(File(10), File(20)), File(5),
# Dir(File(30)), ) Total: 65
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is queried often and files change.
# 2. Hard links: the same file appears under two
# directories.
# 3. Find the largest directory.
#
# INTERVIEW NOTES
# A total over a tree is a post-order sum. Two things
# change the answer in real file systems: depth (recursion
# limit) and sharing or cycles (the same node reachable
# twice must be counted once). A visited set solves both
# the cycle risk and the double counting, and an explicit
# stack solves the depth.
#
# BRUTE FORCE (say it first): Plain recursion
# Sum a file's size, or the sum over a directory's
# children. Correct for ordinary trees and the answer most
# interviews want first. It overflows on very deep trees
# and loops forever on a cycle. Time O(n). Space O(depth)
# call stack.


import heapq


# HELPERS for the brute force
class File:
    def __init__(self, size):
        self.size, self.parent = size, None


# BRUTE FORCE (say it first): Plain recursion | time O(n)
def total_size_recursive(node):
    if isinstance(node, File):
        return node.size
    return sum(
        total_size_recursive(c) for c in node.children
    )
```

```
# HELPERS for follow-up 3
class Dir:
    def __init__(self, *children):
        self.children, self.parent = list(children), None
        # cached total; None means "needs recomputing"
        self.cached = None
        for child in children:
            child.parent = self


def size_of(node):
    # a file's size, or a directory's cached total
    if isinstance(node, File):
        return node.size
    return node.cached


def cached_total(root):
    # post-order: a directory after all its children
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        if isinstance(node, Dir) and node.cached is None:
            stack.extend(node.children)
    for node in reversed(order):
        if isinstance(node, Dir) and node.cached is None:
            kids = node.children
            node.cached = sum(size_of(c) for c in kids)
    return root.cached


# BASE QUESTION (optimal): Iterative traversal with a
# visited set
# Idea: An explicit stack removes the recursion limit, and
# a set of visited node ids makes each node count once, so
# shared directories are not double counted and cycles
# terminate.
# FOLLOW-UP 2: Hard links: the same file appears under two
# directories.
# Answer: Count a file once per identity (a visited set of
# ids, as coded) if the question asks for disk usage, but
# once per path if it asks for the sum of the sizes shown.
# Ask which.
def total_size(root, count_once=True):
```

```
    # iterative walk; count_once handles hard links
    total, seen, stack = 0, set(), [root]
    while stack:
        node = stack.pop()
        # FOLLOW-UP 2: hard links: count a shared file
        # once
        if count_once:
            if id(node) in seen:
                continue
            seen.add(id(node))
        if isinstance(node, File):
            total += node.size
        else:
            stack.extend(node.children)
    return total


# FOLLOW-UP 1: The tree is queried often and files change.
# Answer: Cache each directory's total and, when a file
# changes, clear the cache only for its ancestors (walk
# parent links). A query recomputes only cleared nodes.
# Updates cost O(depth).
def resize(file, size):
    # a file changed: only its ancestors go stale
    file.size = size
    node = file.parent
    while node is not None:
        node.cached = None
        node = node.parent


# FOLLOW-UP 3: Find the largest directory.
# Answer: Compute totals in post-order and keep the
# maximum; it is the same pass. For top-K directories keep
# a heap of size K. Cost: one post-order pass, O(n);
# keeping the top K directories with a heap of size K
# makes it O(n log K).
def directory_totals(root):
    # the size of every directory, in one pass
```

```
    cached_total(root)
    out, stack = [], [root]
    while stack:
        node = stack.pop()
        if isinstance(node, Dir):
            out.append(node.cached)
```

```
            stack.extend(node.children)
    return out


# FOLLOW-UP 3: Find the largest directory.
# Answer: Compute totals in post-order and keep the
# maximum; it is the same pass. For top-K directories keep
# a heap of size K. Cost: one post-order pass, O(n);
# keeping the top K directories with a heap of size K
# makes it O(n log K).
def top_k_directories(root, k):
    # keep a heap of size k: O(n log k)
    return heapq.nlargest(k, directory_totals(root))


if __name__ == "__main__":
    inner = Dir(File(10), File(20))
    root = Dir(inner, File(5), Dir(File(30)))
    assert total_size(root) == 65
    assert cached_total(root) == 65
    # change one file: only its ancestors are recomputed
    first = inner.children[0]
    resize(first, 15)
    assert root.cached is None and inner.cached is None
    assert cached_total(root) == 70
    assert top_k_directories(root, 2) == [70, 35]
    # a hard link: the same file under two directories
    shared = File(7)
    both = Dir(Dir(shared), Dir(shared))
    assert total_size(both) == 7
    assert total_size(both, count_once=False) == 14
```

NOTES FROM CANDIDATES

If the interviewer says "strict tree", drop the visited set and stick with plain DFS — extra cycle handling reads as over-engineering.
For the cache follow-up, store the cached total on each directory node; when a file changes, walk parent pointers invalidating along the path.
The interviewer also pushes for an iterative DFS (explicit stack) after the recursive solution lands — have both ready.

PREPARATION

Drill recursive-and-iterative DFS over directory-style trees.
Have ready: cycle-safe DFS (visited set), memoized DFS (cache on internal nodes), invalidation cascade.
Practice the same problem under the the canonical "longest absolute file path" parsing problem for warm-up.

STUDY HERE: topic 5, Trees and Tries | main topic: Trees

Also fits: Strings, Parsing and Recursion (listed there as SKIP, no pages). Site tags: tree, recursion, parsing, medium

MEDIUM | 1Point3Acres | onsite-coding | 3 of 10

Nested List Weight Sum (LC 339) with Follow-Ups

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK **ALL TIME 16.5** **30 DAYS 0** **PRIORITY 156/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 339 Nested List Weight Sum

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/nested-list-weighted-sum-parsing

QUESTION

EXAMPLES

```
"[3, 8, [2, 14], [2, [91]]]" -> 3×1 + 8×1 + 2×2 + 14×2
    + 2×2 + 91×3 = 320.
Use [2, [91]] as a manual trace for bracket depth and
    the final-number flush.
```

APPROACH

Write DFS and BFS versions over a parsed nested-list interface and explain when recursion depth becomes unsafe.
Build the string parser as a tiny state machine, including multi-digit values and the end-of-input flush.
Dry-run [2, [91]] aloud while tracking depth, buffered number, and accumulated sum.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the input is already parsed or a string, whether negatives and multi-digit numbers appear, and whether weight is depth from the top (1 at the outside) or the inverse weight variant.

BRUTE FORCE TO SAY OUT LOUD

Parse into a tree of lists and recurse with depth. Simple but recursion can overflow.

OPTIMAL IDEA

For a string, scan once with a depth counter: '[' adds one, ']' subtracts one, and an integer is multiplied by the current depth when its last digit ends (flush the number on any non-digit and at the end of the string). For parsed lists use BFS level by level. O(n), O(1) extra for the scanner.

DRY-RUN THESE CASES

[3, 8, [2, 14], [2, [91]]] gives 320. A number glued to the closing bracket. A negative number. A deeply nested list.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

The inverse weight version (LeetCode 364), depth limit, or iterative versus recursive.

PITFALLS

Not flushing a trailing number. Treating '-' as a separate token.

HOW TO THINK ABOUT IT

The weight of a number is just the nesting depth where it appears, and the nesting depth is the number of '[' still open. That means you never need to build the nested structure: scan once with a counter, and flush a number whenever a non-digit arrives (and at the end).

Basis: LeetCode 339, Nested List Weight Sum, with the nested list given as a string.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(text):
    nested = json.loads(text)  # parse the lists
    return depth_sum(nested, 1)
def depth_sum(items, depth):
    total = 0
    for item in items:
        if isinstance(item, int):
            total += item * depth
        else:
            total += depth_sum(item, depth + 1)
    return total
```

TIME	SPACE
O(n)	O(depth)

WHY IT IS SLOW

It builds the whole nested structure and recurses as deep as the nesting, so memory and the call stack grow with depth. A single left-to-right scan with a bracket counter needs O(1) extra space.

OPTIMAL SOLUTION · Single pass over the string with a depth counter

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 When does the recursive (DFS) version become unsafe?

Python's default recursion limit is about 1000 frames, so an input nested 10,000 levels deep raises RecursionError. DFS also uses O(depth) call-stack space. The single-pass scanner uses O(1) and BFS uses O(widest level); both avoid the problem. In Java the same limit appears as StackOverflowError.

FOLLOW-UP 2 What is the inverse version (deeper integers weigh less)?

That is LeetCode 364. Do a level-by-level pass that keeps a running sum of everything seen so far and add it to the total once per level: total += running_sum on each level. Shallow integers are added many times, deep ones once, which equals weight (maxDepth + 1 - depth).

FOLLOW-UP 3 The inverse version (shallow integers weigh less, LeetCode 364).

Process level by level and keep a running sum of every integer seen so far; add the running sum to the total once per level that holds an integer or lies above one (trim trailing levels that contain only empty lists). Shallow numbers are added at every deeper level, which equals the weight maxDepth - depth + 1.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Nested List Weight Sum (LC 339) with
# Follow-Ups
#
# Examples: "[3, 8, [2, 14], [2, [91]]]" -> 3×1 + 8×1 + 2×
2
# + 14×2 + 2×2 + 91×3 = 320. Use [2, [91]] as a manual
# trace for bracket depth and the final-number flush.
#
# FOLLOW-UPS (answered below, in this order):
# 1. When does the recursive (DFS) version become unsafe?
# 2. What is the inverse version (deeper integers weigh
# less)?
# 3. The inverse version (shallow integers weigh less,
# LeetCode 364).
#
# INTERVIEW NOTES
# The weight of a number is just the nesting depth where
# it appears, and the nesting depth is the number of '['
# still open. That means you never need to build the
# nested structure: scan once with a counter, and flush a
# number whenever a non-digit arrives (and at the end).
#
# BRUTE FORCE (say it first): Parse the string into lists,
# then recurse with the depth
# json.loads turns the text into nested lists; a recursive
# function adds each integer times its depth. Clear, but
# it builds the whole structure and recurses as deep as
# the nesting. Time O(n). Space O(depth).
```

```
import json


# BRUTE FORCE (say it first): Parse the string into lists,
# then recurse with the depth | time O(n)
def depth_sum_parsed(s):
    def go(items, depth):
        return sum(
            (
                x * depth
                if isinstance(x, int)
                else go(x, depth + 1)
            )
            for x in items
        )
```

```
    return go(json.loads(s), 1)


# HELPERS for the base question
def with_depths(nested):
    # every integer with its depth (top level = 1),
    # using a stack so deep input cannot overflow
    stack = [(nested, 1)]
    while stack:
        item, depth = stack.pop()
        if isinstance(item, list):
            for child in item:
                stack.append((child, depth + 1))
        else:
            yield item, depth - 1


# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends.
def weight_sum(nested):
    # deeper integers weigh more (LeetCode 339)
    return sum(v * d for v, d in with_depths(nested))


# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends.
def weight_sum_string(s):
    # one pass over the text, O(1) extra space
    depth = total = 0
    num, sign, have = 0, 1, False
    # a closing sentinel flushes a trailing number
    for ch in s + "]":
        if ch.isdigit():
            num, have = num * 10 + int(ch), True
            continue
        # any other character ends the number being read
        if have:
```

```
            total += sign * num * depth
            num, sign, have = 0, 1, False
        if ch == "[":
            depth += 1
        elif ch == "]":
            depth -= 1
        elif ch == "-":
            sign = -1
    return total
```

```
# FOLLOW-UP 2: What is the inverse version (deeper
# integers weigh less)?
# Answer: That is LeetCode 364. Do a level-by-level pass
# that keeps a running sum of everything seen so far and
# add it to the total once per level: total += running_sum
# on each level.
# FOLLOW-UP 3: The inverse version (shallow integers weigh
# less, LeetCode 364).
# Answer: Process level by level and keep a running sum of
# every integer seen so far; add the running sum to the
# total once per level that holds an integer or lies above
# one (trim trailing levels that contain only empty
# lists).
def inverse_weight_sum(nested):
    # deeper integers weigh less (LeetCode 364)
    pairs = list(with_depths(nested))
    if not pairs:
        return 0
    deepest = max(d for _, d in pairs)
    # weight = deepest + 1 - depth
    return sum(v * (deepest + 1 - d) for v, d in pairs)


if __name__ == "__main__":
    data = [3, 8, [2, 14], [2, [91]]]
    # 3 + 8 + 2*2 + 14*2 + 2*2 + 91*3 = 320
    assert weight_sum(data) == 320
    text = "[3, 8, [2, 14], [2, [91]]]"
    assert weight_sum_string(text) == 320
    assert weight_sum_string("[2, [91]]") == 2 + 91 * 2
    # inverse weights: [[1,1],2,[1,1]] -> 8
    assert inverse_weight_sum([[1, 1], 2, [1, 1]]) == 8
    assert inverse_weight_sum([1, [4, [6]]]) == 17
    # empty lists add nothing
```

```
    assert weight_sum([[], [[]]]) == 0
    # 20000 levels deep is still fine
    deep = []
    for _ in range(20000):
        deep = [deep, 1]
    assert weight_sum(deep) > 0
```

NOTES FROM CANDIDATES

The easy-to-miss bug is leaving the last buffered number uncommitted when the string ends. Manually tracing the small nested case exposed this during the interview, and fixing it proactively was part of the signal.
DFS visits each nested value once in O(n) time and uses O(d) call-stack space at maximum depth d; BFS is also O(n) time with a frontier proportional to the widest level. In the string parser, track the sign and whether a number is currently buffered, flush only when that flag is set, and flush before decrementing depth on].

PREPARATION

Write DFS and BFS versions over a parsed nested-list interface and explain when recursion depth becomes unsafe.
Build the string parser as a tiny state machine, including multi-digit values and the end-of-input flush.
Dry-run [2, [91]] aloud while tracking depth, buffered number, and accumulated sum.

STUDY HERE: topic 5, Trees and Tries | main topic: Trees

Also fits: Graphs and Search, Strings, Parsing and Recursion (listed there as SKIP, no pages). Site tags: tree, dfs, recursion, medium

MEDIUM | 1Point3Acres | phone-screen | 4 of 10

Longest Consecutive Path in Binary Tree

ASKED 2026-07-02 | 110 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 16.5** **30 DAYS 0** **PRIORITY 157/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #549 Binary Tree Longest Consecutive Sequence II

LeetCode match: LC 549 Binary Tree Longest Consecutive Sequence II (related)

Frequency: Single report | Last asked: 2026-07-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/binary-tree-longest-consecutive-path

QUESTION

Given a binary tree, return the length of the longest parent-to-child path whose values increase consecutively by 1. Follow-up: allow the path to extend in any direction through the tree, not only from parent to child.SWEtreedfsrecursionmediumFrequencySingle reportLast asked2026-07-02Stagephone-screen

APPROACH

Write the parent-to-child traversal in under 10 minutes with a global best.
Practice the bidirectional tree follow-up where each node returns two lengths upward.
Dry-run single-node, duplicate-value, and split-branch cases before discussing complexity.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether length counts nodes or edges, whether the path goes only parent to child (base) or can bend through a node (follow-up), and whether the sequence must increase by exactly one.

BRUTE FORCE TO SAY OUT LOUD

From every node walk down both children recording the longest run. O(n^2) on a skewed tree.

OPTIMAL IDEA

Base: DFS passing the current run length; it grows by one when the child is exactly the parent plus one, otherwise it resets to 1; keep the global best. O(n). Follow-up: each node returns the longest increasing and longest decreasing chain going down from it; the best path through a node is inc + dec - 1.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

A single node, an empty tree, duplicate values, a split branch, a descending chain.

LIKELY FOLLOW-UPS

The any-direction version (above), and an iterative version for very deep trees.

PITFALLS

Counting edges when the interviewer expects nodes. Resetting the run to 0 instead of 1.

HOW TO THINK ABOUT IT

Each node needs to know the longest run that ends at its parent, so pass the current length down and reset it to 1 when the value is not parent + 1. For the bending version each node must report two lengths upward (increasing and decreasing chains going down), and the best path through a node combines one of each.

Basis: LeetCode 298 (parent-to-child) and LeetCode 549 (any direction), Binary Tree Longest Consecutive Sequence I and II.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def walk(node):
    best = 1
    for child in (node.left, node.right):
        if child and child.val == node.val + 1:
            best = max(best, 1 + walk(child))
    return best
answer = 0
for node in all_nodes(root):
    answer = max(answer, walk(node))
```

TIME	SPACE
O(n * h)	O(h)

WHY IT IS SLOW

A node deep in a chain is walked again from every ancestor, O(n * h). Passing the current run length down in one traversal makes it O(n).

OPTIMAL SOLUTION · Top-down DFS carrying the current run

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What does the any-direction version return for a single node, or for duplicate values?

A single node is a path of length 1 (0 edges), and an empty tree is 0. Equal values never extend a chain because the step must be exactly +1 or -1, so duplicates reset both counters to 1 at that node.

FOLLOW-UP 2 Can you avoid recursion?

Yes: do an iterative post-order or a BFS that stores (node, run) pairs for the top-down version. The bending version needs child results before the parent, so use an explicit stack with a post-order visit or process nodes in reverse BFS order using a dict from node to (inc, dec). It matters because a chain-shaped tree makes the recursive depth n.

FOLLOW-UP 3 Return the path itself, not only its length.

Carry the current run as a list (or its start node and length) in the traversal and copy it into the best answer when it becomes longer.

FOLLOW-UP 4 The path may go in any direction (LeetCode 549).

Return the pair (longest increasing chain down, longest decreasing chain down) from each node, and update the global best with inc + dec - 1 at the node. Verified against exhaustive path enumeration for small trees.

FOLLOW-UP 5 The values must increase by 1 or decrease by 1 along the path but not mix.

That is exactly the any-direction version: the increasing and decreasing parts meet at the bend; a path cannot zig-zag between increasing and decreasing steps.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Longest Consecutive Path in Binary Tree
#
# Given a binary tree, return the length of the longest
# parent-to-child path whose values increase consecutively
# by 1. Follow-up: allow the path to extend in any
# direction through the tree, not only from parent to
# child.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What does the any-direction version return for a
# single node, or for duplicate values?
# 2. Can you avoid recursion?
# 3. Return the path itself, not only its length.
# 4. The path may go in any direction (LeetCode 549).
# 5. The values must increase by 1 or decrease by 1 along
# the path but not mix.
#
# INTERVIEW NOTES
# Each node needs to know the longest run that ends at its
# parent, so pass the current length down and reset it to
# 1 when the value is not parent + 1. For the bending
# version each node must report two lengths upward
# (increasing and decreasing chains going down), and the
# best path through a node combines one of each.
#
# BRUTE FORCE (say it first): Start a downward walk from
# every node
# For each node, walk down both children while the next
# value is exactly one more, tracking the longest run.
# Correct and the oracle for the base question; nodes deep
# in a chain are re-walked from every ancestor. Time O(n *
# h). Space O(h).


# BRUTE FORCE (say it first): Start a downward walk from
# every node | time O(n * h)
def longest_consecutive_brute(root):
    best = 0

    def down(node):
        a = b = 0
        if node.left and node.left.val == node.val + 1:
            a = down(node.left)
        if (
            node.right
            and node.right.val == node.val + 1
        ):
            b = down(node.right)
        return 1 + max(a, b)

    def visit(node):
        nonlocal best
        if node:
            best = max(best, down(node))
            visit(node.left)
            visit(node.right)

    visit(root)
    return best


# BASE QUESTION (optimal): Top-down DFS carrying the
# current run
# Idea: Pass the length of the run ending at the parent.
# If the child is exactly parent + 1 the run grows,
# otherwise it restarts at 1. Track the global best.
# Length is counted in nodes; state that choice out loud.
def longest_consecutive(root, want_path=False):
    # parent -> child path whose values go up by exactly 1
    best, best_path = 0, []
    stack = [(root, None, [])] if root else []
    while stack:
        node, parent, run = stack.pop()
        # extend the run or start a new one at this node
        if parent is not None and node.val == parent + 1:
            run = run + [node.val]
        else:
            run = [node.val]
        if len(run) > best:
            best, best_path = len(run), run
        for child in (node.left, node.right):
            if child is not None:
                stack.append((child, node.val, run))
    # FOLLOW-UP 3: want_path=True returns the path itself
    return best_path if want_path else best


# FOLLOW-UP 4: The path may go in any direction (LeetCode
```

```
# 549).
# Answer: Return the pair (longest increasing chain down,
```

```
# longest decreasing chain down) from each node, and
# update the global best with inc + dec - 1 at the node.
# Verified against exhaustive path enumeration for small
# trees.
def longest_any_direction(root):
    # LeetCode 549: the path may bend at one node
    if root is None:
        return 0
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                stack.append(child)
    # inc[x] / dec[x]: longest chain going down from x
    inc, dec, best = {}, {}, 1
    # children are processed before their parents
    for node in reversed(order):
        up = down = 1
        for child in (node.left, node.right):
            if child is None:
                continue
            if child.val == node.val + 1:
                up = max(up, inc[child] + 1)
            if child.val == node.val - 1:
                down = max(down, dec[child] + 1)
        inc[node], dec[node] = up, down
        # the bend joins both chains (counted node once)
        best = max(best, up + down - 1)
    return best


# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


if __name__ == "__main__":
    #     1
    #    / \
    #   2   0
    #  /
    # 3
```

```
    left = TreeNode(2, TreeNode(3))
    tree = TreeNode(1, left, TreeNode(0))
    assert longest_consecutive(tree) == 3
    assert longest_consecutive(tree, True) == [1, 2, 3]
    # bending: 2 - 1 - 0 or 3 - 2 - 1 - 0 is length 4
    assert longest_any_direction(tree) == 4
    assert longest_consecutive(None) == 0
    assert longest_any_direction(None) == 0
    # duplicates never extend a chain
    dup = TreeNode(5, TreeNode(5), TreeNode(5))
    assert longest_consecutive(dup) == 1
    assert longest_any_direction(dup) == 1
    # a long chain needs no recursion
    chain = TreeNode(0)
    cur = chain
    for i in range(1, 5000):
        cur.left = TreeNode(i)
        cur = cur.left
    assert longest_consecutive(chain) == 5000
```

NOTES FROM CANDIDATES

In the base version, carry the current consecutive length while traversing downward and reset to 1 whenever the child is not exactly parent + 1.
For the any-direction follow-up, track increasing and decreasing chain lengths at each node, then combine through the node when the left/right child values support a consecutive bridge.
Clarify whether the answer is measured in nodes or edges before coding; Google candidates usually state node count.

PREPARATION

Write the parent-to-child traversal in under 10 minutes with a global best.
Practice the bidirectional tree follow-up where each node returns two lengths upward.
Dry-run single-node, duplicate-value, and split-branch cases before discussing complexity.

STUDY HERE: topic 5, Trees and Tries | main topic: Trees

Also fits: Arrays, Hashing and Two Pointers, Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: tree, hashmap, object-design, medium

MEDIUM | 1Point3Acres | onsite-coding | 5 of 10

Employee Hierarchy Add / Delete

ASKED 2026-06-10 | 132 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 227/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/employee-hierarchy-add-delete

QUESTION

Design an employee hierarchy class that supports adding employees under managers and deleting employees by id. Follow-ups cover deleting managers, deleting the CEO, cycle handling, and choosing a replacement manager by seniority.SWEtreehashmapobject-designmediumFrequencySingle reportLast asked2026-06-10Stageonsite-coding

APPROACH

Implement tree mutation operations with parent pointers. Practice deleting an internal node while reconnecting children. Add cycle checks with DFS or parent-chain walking before accepting a new edge.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what happens when a manager is deleted (promote a report, reassign to the grandparent, or reject), what happens when the CEO is deleted, whether adding can create a cycle, and how seniority is defined.

BRUTE FORCE TO SAY OUT LOUD

Store parent pointers only and search the whole tree to find children on every delete. O(n).

OPTIMAL IDEA

Keep id to node, and for each node a parent pointer and a children set. Add: validate the manager exists and the new id is free. Delete: take the children of the removed node and attach them to a replacement (the deleted node's parent, or a chosen successor), update pointers, and remove it from its parent's children. CEO needs a stated policy. Compare candidates by (level descending, insertion order ascending) for the seniority rule.

DRY-RUN THESE CASES

Delete a leaf, a middle manager, the CEO, an unknown id, adding with a missing manager.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Cycle detection by walking up parent pointers, bulk deletes, or a history of changes.

PITFALLS

Leaving dangling children references. Not choosing a policy for the root.

HOW TO THINK ABOUT IT

A hierarchy is a tree you update, so store links in both directions (parent per node, children per node). Then every operation touches only the neighbours of one node instead of searching the tree. For each delete case (leaf, middle, root) say the policy first, because the data structure is easy once the rule is fixed.

Basis: Written. Assumed policies (confirm them): deleting a manager hands their reports to that manager's own manager; deleting the CEO is allowed only if exactly one report exists (that report becomes CEO), otherwise it is rejected; ids are unique.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
manager = {}  # employee -> manager
def delete(e):
    reports = [x for x in manager
               if manager[x] == e]
    for x in reports:
        manager[x] = manager[e]
    del manager[e]
```

TIME	SPACE
O(n) per delete	O(n)

WHY IT IS SLOW

Every delete scans all employees, O(n). Keeping a children set per employee makes it O(number of reports).

OPTIMAL SOLUTION · Parent map plus children sets

TIME	SPACE
O(r) per delete where r is the number of reports	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support moving an employee under another manager and reject cycles.

Walk up from the proposed new manager through manager pointers; if you meet the employee being moved, the move would put a node under its own descendant, so reject it. O(depth).

FOLLOW-UP 2 When a manager leaves, promote the most senior report instead of the grandparent.

Store a hire order (insertion counter) per employee and pick the report with the smallest one: the new manager takes over the other reports. Say how you break ties.

FOLLOW-UP 3 The CEO leaves.

Pick a policy and say it: reject, promote the most senior report (the rest report to them), or leave the company rootless and hold a set of roots. The structure above needs only a promoted root.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Employee Hierarchy Add / Delete
#
# Design an employee hierarchy class that supports adding
# employees under managers and deleting employees by id.
# Follow-ups cover deleting managers, deleting the CEO,
# cycle handling, and choosing a replacement manager by
# seniority.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Support moving an employee under another manager and
# reject cycles.
# 2. When a manager leaves, promote the most senior report
# instead of the grandparent.
# 3. The CEO leaves.
#
# INTERVIEW NOTES
# A hierarchy is a tree you update, so store links in both
# directions (parent per node, children per node). Then
# every operation touches only the neighbours of one node
# instead of searching the tree. For each delete case
# (leaf, middle, root) say the policy first, because the
# data structure is easy once the rule is fixed.
#
# BRUTE FORCE (say it first): Store only parent pointers
# and search for children
# Keep a dict from employee to manager. To delete, scan
# all entries to find the reports and reassign them.
# Correct, but every delete is a full scan. Time O(n) per
# delete. Space O(n).


# BRUTE FORCE (say it first): Store only parent pointers
# and search for children | time O(n) per delete
class OrgBrute:
    def __init__(self, ceo):
        self.manager = {ceo: None}

    def add(self, emp, mgr):
        if (
            emp in self.manager
            or mgr not in self.manager
        ):
            return False
        self.manager[emp] = mgr
        return True
```

```
    def delete(self, emp):
        if emp not in self.manager:
            return False
        mgr = self.manager[emp]
        reports = [
            e
            for e, m in self.manager.items()
            if m == emp
        ]
        if mgr is None and len(reports) != 1:
            return False
        for r in reports:
            self.manager[r] = mgr
        del self.manager[emp]
        return True


# BASE QUESTION (optimal): Parent map plus children sets
# Idea: Keep manager[emp] and reports[emp] (a set). Add
# inserts into both. Delete reattaches each report to the
# deleted node's manager and removes the node from its
# manager's set.
# FOLLOW-UP 1: Support moving an employee under another
# manager and reject cycles.
# Answer: Walk up from the proposed new manager through
# manager pointers; if you meet the employee being moved,
# the move would put a node under its own descendant, so
# reject it. O(depth).
class Org:
    def __init__(self, ceo):
        self.root = ceo
        self.manager = {ceo: None}
        self.reports = {ceo: set()}
        # hire order, used for "most senior" choices
        self.hired = {ceo: 0}
        self.clock = 1

    def add(self, emp, mgr):
        if emp in self.manager or mgr not in self.manager:
            return False
        self.manager[emp] = mgr
        self.reports[emp] = set()
        self.reports[mgr].add(emp)
        self.hired[emp] = self.clock
```

```
        self.clock += 1
        return True

    def senior(self, people):
        # the person hired earliest
        return min(people, key=lambda p: self.hired[p])

    def delete(self, emp, policy="grandparent"):
        if emp not in self.manager:
            return False
        mgr = self.manager[emp]
        kids = set(self.reports[emp])
        # FOLLOW-UP 3: the CEO leaves
        if mgr is None and not kids:
            # the CEO alone: nothing could take over
            return False
        # FOLLOW-UP 2: promote the most senior report
        if policy == "senior" or mgr is None:
            # the most senior report takes the place of
            # emp
            heir = self.senior(kids) if kids else None
        else:
            heir = mgr
        for r in kids:
            if r != heir:
                self.reassign(r, heir)
        if mgr is None:
            # the promoted report becomes the new CEO
            self.root = heir
```

```
            self.manager[heir] = None
        elif policy == "senior" and heir != mgr:
            self.reassign(heir, mgr)
        if mgr is not None:
            self.reports[mgr].discard(emp)
        del self.manager[emp], self.reports[emp]
        return True

    def reassign(self, emp, new_mgr):
        old = self.manager[emp]
        if old is not None and old in self.reports:
            self.reports[old].discard(emp)
        self.manager[emp] = new_mgr
        self.reports[new_mgr].add(emp)

    # FOLLOW-UP 1: move an employee, reject cycles
```

```
    def move(self, emp, new_mgr):
        # walk up from the new manager: meeting emp is a
        # cycle
        node = new_mgr
        while node is not None:
            if node == emp:
                return False
            node = self.manager[node]
        self.reassign(emp, new_mgr)
        return True


if __name__ == "__main__":
    o = Org("ceo")
    for e, m in [("a", "ceo"), ("b", "a"), ("c", "a")]:
        assert o.add(e, m)
    assert not o.add("a", "ceo")
    assert not o.add("x", "nobody")
    # deleting a manager: reports go to the grandparent
    assert o.delete("a")
    assert o.manager["b"] == "ceo"
    assert o.manager["c"] == "ceo"
    # a move that would create a cycle is rejected
    o.add("d", "b")
    assert not o.move("b", "d") and o.move("d", "c")
    # promote the most senior report instead
    p = Org("ceo")
    for e, m in [("m", "ceo"), ("x", "m"), ("y", "m")]:
        p.add(e, m)
    assert p.delete("m", "senior")
    assert p.manager["x"] == "ceo"
    assert p.manager["y"] == "x"
    # the CEO leaves: the most senior report takes over
    q = Org("ceo")
    q.add("a", "ceo")
    q.add("b", "ceo")
    assert q.delete("ceo")
    assert q.root == "a" and q.manager["b"] == "a"
    assert not Org("solo").delete("solo")
```

NOTES FROM CANDIDATES

Use maps from id to node and id to children; keep parent pointers so deletes do not require a full traversal.
CEO deletion needs an explicit policy: reject, promote a child, or create a new root. State the policy and keep it consistent.
For the seniority follow-up, maintain insertion order on nodes and compare (level desc, insertion_order asc).

PREPARATION

Implement tree mutation operations with parent pointers.
Practice deleting an internal node while reconnecting children.
Add cycle checks with DFS or parent-chain walking before accepting a new edge.

STUDY HERE: topic 5, Trees and Tries | main topic: Trees

Also fits: Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: data-structure, tree, medium

MEDIUM | IN APP LC 703 | 1Point3Acres | onsite-coding | 6 of 10

★ TOP 75: rank 68 in the Medium and Hard priority list

★ Streaming Insert + Find K-th Largest

ASKED 2026-02-04 | 258 DAYS AGO | OLDER

STUDY: TALK **ALL TIME 32.6** **30 DAYS 0** **PRIORITY 118/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: your app's interview approach (LeetCode #703)

LeetCode match: LC 703 Kth Largest Element in a Stream

Frequency: Single report | Last asked: 2026-02-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/insert-find-kth-largest

QUESTION

EXAMPLES

```
insert(3)
insert(3)
insert(2)
findLargest(0) -> 3
findLargest(1) -> 3
findLargest(2) -> 2
```

APPROACH

Sketch the size-augmented BST node (left subtree count + duplicate count) on paper before the interview.
Practice the Fenwick-tree order-statistics trick (find k-th by binary search on prefix sums).
Be ready to defend the chosen data structure: balanced BST is clean but verbose; Fenwick is short but assumes bounded values.

HOW TO THINK ABOUT IT

If k can change from query to query you need every value, in a structure that supports rank queries; if k is fixed you only need the k largest, and a min-heap of size k gives the answer at its root. Ask which of the two it is: it decides everything.

Basis: LeetCode 703, Kth Largest Element in a Stream, generalized so k is passed to each query (0-indexed in the examples).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
items = []
def insert(x):
    items.append(x)
def find_largest(k):
    ordered = sorted(items, reverse=True)
    return ordered[k]
```

BRUTE FORCE

TIME	SPACE
O(n log n) per query	**O(n)**

WHY IT IS SLOW

Every query re-sorts the whole history, O(n log n). A sorted list with binary search makes each query O(1) after an O(log n) search plus shift; for a fixed k a size-k heap is O(log k).

OPTIMAL SOLUTION · Sorted list with bisect (k chosen per query)

TIME	SPACE
O(log n) search + O(n) shift per insert, O(1) query	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if values can also be deleted?

The heap can no longer answer because removing an arbitrary element is O(k). Keep a sorted structure that supports delete: a balanced BST or sorted container with order statistics, or a Fenwick tree over compressed values (add -1 to delete). Both give O(log n) insert, delete and k-th.

FOLLOW-UP 2 What is the trade-off between the two designs?

The heap is the smallest and fastest when k is fixed and small (O(log k), O(k) memory) but cannot change k or delete. The sorted list answers any k and is simplest, at O(n) memory and a linear shift per insert. Choose from the requirements, and say which assumption you made.

FOLLOW-UP 3 Values can also be deleted.

A heap cannot delete an arbitrary value efficiently. Keep a sorted structure: a sorted list with bisect (O(n) per delete because of shifting) or a balanced tree / Fenwick tree over compressed values (O(log n) for insert, delete and k-th).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Streaming Insert + Find K-th Largest
#
# Examples: insert(3) insert(3) insert(2) findLargest(0) -
>
# 3 findLargest(1) -> 3 findLargest(2) -> 2
#
# FOLLOW-UPS (answered below, in this order):
```

```
# 1. What if values can also be deleted?
# 2. What is the trade-off between the two designs?
# 3. Values can also be deleted.
#
# INTERVIEW NOTES
# If k can change from query to query you need every
# value, in a structure that supports rank queries; if k
# is fixed you only need the k largest, and a min-heap of
# size k gives the answer at its root. Ask which of the
# two it is: it decides everything.
#
# BRUTE FORCE (say it first): Store everything and sort
# for every query
# Append values to a list; on a query sort it in
# descending order and read index k. Correct and the
# oracle; each query re-sorts everything. Time O(n log n)
# per query. Space O(n).


import heapq
from bisect import bisect_left, insort


# BRUTE FORCE (say it first): Store everything and sort
# for every query | time O(n log n) per query
class StreamKthBrute:
    def __init__(self):
        self.a = []

    def insert(self, x):
        self.a.append(x)

    def findLargest(self, k):
        s = sorted(self.a, reverse=True)
        return s[k] if 0 <= k < len(s) else None


# BASE QUESTION (optimal): Sorted list with bisect (k
# chosen per query)
```

```
# Idea: Keep all values in an ascending list with insort.
# The k-th largest (0-indexed as in the examples) is a[-1
# - k].
# FOLLOW-UP 1: What if values can also be deleted?
# Answer: The heap can no longer answer because removing
# an arbitrary element is O(k).
# FOLLOW-UP 3: Values can also be deleted.
# Answer: A heap cannot delete an arbitrary value
# efficiently.
class StreamKth:
    """Any k, with deletes: a sorted list."""

    def __init__(self):
        self.a = []

    def insert(self, x):
        # keep the list sorted: O(log n) search, O(n)
        # shift
        insort(self.a, x)

    # FOLLOW-UP 1, 3: values can also be deleted
    def delete(self, x):
        # remove one copy of x if present
        i = bisect_left(self.a, x)
        if i < len(self.a) and self.a[i] == x:
            del self.a[i]
            return True
        return False

    def find_largest(self, k):
        # k = 0 is the largest value
        if 0 <= k < len(self.a):
            return self.a[-1 - k]
        return None


# FOLLOW-UP 2: What is the trade-off between the two
# designs?
# Answer: The heap is the smallest and fastest when k is
# fixed and small (O(log k), O(k) memory) but cannot
# change k or delete. The sorted list answers any k and is
# simplest, at O(n) memory and a linear shift per insert.
class FixedKth:
    """Fixed k, inserts only: a min-heap of the best k."""

    def __init__(self, k):
        self.k, self.heap = k, []

    def insert(self, x):
        heapq.heappush(self.heap, x)
        # keep only the k largest values
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)

    def kth_largest(self):
        # the smallest of the k largest
        if len(self.heap) < self.k:
            return None
        return self.heap[0]


if __name__ == "__main__":
    s = StreamKth()
    for x in (3, 3, 2):
        s.insert(x)
    top3 = [s.find_largest(k) for k in range(3)]
    assert top3 == [3, 3, 2]
    assert s.find_largest(3) is None
    # deletes work too
    assert s.delete(3) and not s.delete(9)
    assert [s.find_largest(k) for k in range(2)] == [3, 2]
    f = FixedKth(2)
    assert f.kth_largest() is None
    for x in (5, 1, 9, 7):
        f.insert(x)
    # the two largest are 9 and 7, so the 2nd largest is 7
    assert f.kth_largest() == 7
```

NOTES FROM CANDIDATES

An order-statistics structure (size-augmented BST / balanced tree / skiplist) gives O(log n) for both ops.
An order-statistics Fenwick tree over a coordinate-compressed value range gives O(log V).
A naive sorted array gives O(log n) find but O(n) insert; mention as a baseline.
Two heaps (the canonical streaming-median pattern) only work for a fixed k; the interviewer wants arbitrary k per call.

PREPARATION

Sketch the size-augmented BST node (left subtree count + duplicate count) on paper before the interview.
Practice the Fenwick-tree order-statistics trick (find k-th by binary search on prefix sums).
Be ready to defend the chosen data structure: balanced BST is clean but verbose; Fenwick is short but assumes bounded values.

STUDY HERE: topic 5, Trees and Tries | main topic: Trees

Also fits: Heaps, Stacks and Queues, Greedy and Math (listed there as SKIP, no pages). Site tags: tree, heap, greedy, hard

HARD | 1Point3Acres | onsite-coding | 7 of 10

Huffman Tree Construction

ASKED 2026-06-01 | 141 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 189/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-06-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/huffman-tree-construction

QUESTION

EXAMPLES

```
{"a": 5, "b": 9, "c": 12, "d": 13, "e": 16, "f": 45}
    -> standard Huffman tree with f near the root.
```

APPROACH

Write the heap-based Huffman build from memory in under 15 min.
Drill the encode + decode helpers (DFS to build the code map; tree walk to decode).
Be ready to compare against arithmetic coding (better compression, costlier).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether to return the tree, a code table, or the encoded message, how ties are broken, and what to do with one symbol.

BRUTE FORCE TO SAY OUT LOUD

Repeatedly scan for the two smallest weights. O(n^2).

OPTIMAL IDEA

Min-heap of nodes keyed by (weight, counter). Pop two, merge under a new node with the summed weight, push back, until one node remains. Walk the tree assigning '0' to left and '1' to right. O(n log n). Decode by walking the tree bit by bit. The counter prevents comparing nodes on ties.

DRY-RUN THESE CASES

The CLRS example (total cost 224, 'f' gets a one-bit code). One symbol (code '0'). Equal weights.

LIKELY FOLLOW-UPS

Canonical codes, decoding, or the two-queue O(n) method on sorted input.

PITFALLS

Comparing tree nodes on equal weights. Forgetting the single symbol case.

HOW TO THINK ABOUT IT

The two rarest symbols belong at the deepest level as siblings, so merge them first, treat the merged node as a new symbol with the combined weight, and repeat. 'Take the two smallest' is the job of a min-heap; tie-breaking with a counter keeps the heap from comparing tree nodes.

Basis: Classic greedy algorithm (CLRS 16.3); related to LeetCode 1167, Minimum Cost to Connect Sticks.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(weights):
    items = list(weights)
    cost = 0
    while len(items) > 1:
        items.sort()
        a, b = items[0], items[1]  # two smallest
        items = items[2:] + [a + b]
        cost += a + b
    return cost
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Finding the two smallest by scanning costs O(n) per merge and n - 1 merges, O(n^2). A min-heap does each merge in O(log n).

OPTIMAL SOLUTION · Min-heap merge of the two lightest nodes

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is the greedy choice optimal?

Exchange argument: in some optimal tree the two least frequent symbols are siblings at the deepest level (swapping a shallower, more frequent symbol with a deeper, less frequent one never increases cost). Merging them into one symbol reduces the problem to n - 1 symbols with the same optimal-substructure property, so induction closes it.

FOLLOW-UP 2 How do you make the code canonical, and why would you?

Keep only each symbol's code length from the tree, sort symbols by (length, symbol), and assign consecutive binary codes. A decoder then needs just the lengths rather than the whole tree, which is how DEFLATE stores its tables.

FOLLOW-UP 3 Time complexity if symbols arrive already sorted?

Two-queue method: one queue holds the sorted leaves, the other the merged nodes (which are produced in nondecreasing order). Always pop the smaller front of the two queues. That is O(n) after sorting.

FOLLOW-UP 4 Make the codes canonical so only the code lengths need to be stored.

Sort symbols by (code length, symbol) and assign consecutive binary numbers, shifting left when the length grows. A decoder rebuilds the codes from the lengths alone (this is how DEFLATE stores its tables).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Huffman Tree Construction
#
# Examples: {"a": 5, "b": 9, "c": 12, "d": 13, "e": 16,
# "f": 45} -> standard Huffman tree with f near the root.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is the greedy choice optimal?
# 2. How do you make the code canonical, and why would
# you?
# 3. Time complexity if symbols arrive already sorted?
# 4. Make the codes canonical so only the code lengths
# need to be stored.
#
# INTERVIEW NOTES
# The two rarest symbols belong at the deepest level as
# siblings, so merge them first, treat the merged node as
# a new symbol with the combined weight, and repeat. 'Take
# the two smallest' is the job of a min-heap; tie-breaking
# with a counter keeps the heap from comparing tree nodes.
#
# BRUTE FORCE (say it first): Scan for the two smallest
# weights on every merge
# Keep a list of (weight, tree) items; each round scan for
# the two smallest, merge, and put the result back. The
# same greedy as the heap version, with a linear scan
# instead of a heap operation. Returns the total cost (sum
# of weight * code length). Time O(n^2). Space O(n).


import heapq


# BRUTE FORCE (say it first): Scan for the two smallest
# weights on every merge | time O(n^2)
def huffman_cost_scan(weights):
    items = list(weights)
    cost = 0
    while len(items) > 1:
        items.sort()
        a, b = items[0], items[1]
        cost += a + b
        items = items[2:] + [a + b]
    return cost
```

```
# HELPERS for the base question
def huffman_lengths(freq):
    # code length of every symbol (depth in the tree)
    if not freq:
        return {}
    if len(freq) == 1:
        # a lone symbol still needs one bit
        return {next(iter(freq)): 1}
    # heap entries: (weight, tie-break, symbols below)
    heap = [(w, i, [s]) for i, (s, w) in
            enumerate(sorted(freq.items()))]
    heapq.heapify(heap)
    depth = {s: 0 for s in freq}
    tie = len(heap)
    while len(heap) > 1:
        # merge the two lightest; everything below goes
        # deeper
        w1, _, a = heapq.heappop(heap)
        w2, _, b = heapq.heappop(heap)
        for s in a + b:
            depth[s] += 1
        heapq.heappush(heap, (w1 + w2, tie, a + b))
        tie += 1
    return depth


# FOLLOW-UP 2: How do you make the code canonical, and why
# would you?
# Answer: Keep only each symbol's code length from the
# tree, sort symbols by (length, symbol), and assign
# consecutive binary codes. A decoder then needs just the
# lengths rather than the whole tree, which is how DEFLATE
# stores its tables.
# FOLLOW-UP 4: Make the codes canonical so only the code
# lengths need to be stored.
# Answer: Sort symbols by (code length, symbol) and assign
# consecutive binary numbers, shifting left when the
# length grows. A decoder rebuilds the codes from the
# lengths alone (this is how DEFLATE stores its tables).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def canonical_codes(lengths):
    # assign consecutive binary numbers by (length,
    # symbol)
    code, prev, out = 0, 0, {}
```

```
    for sym, length in sorted(lengths.items(),
                              key=lambda x: (x[1], x[0])):
        # moving to a longer length shifts the code left
        code <<= length - prev
        out[sym] = format(code, f"0{length}b")
        code += 1
        prev = length
    return out


# BASE QUESTION (optimal): Min-heap merge of the two
# lightest nodes
# Idea: Put every symbol in a min-heap keyed by frequency,
# repeatedly pop the two lightest nodes, merge them under
# a new internal node with the summed weight and push it
# back. The last node is the root.
def huffman_codes(freq):
    return canonical_codes(huffman_lengths(freq))


# FOLLOW-UP 3: Time complexity if symbols arrive already
# sorted?
# Answer: Two-queue method: one queue holds the sorted
# leaves, the other the merged nodes (which are produced
# in nondecreasing order). Always pop the smaller front of
# the two queues. That is O(n) after sorting.
def lengths_two_queues(sorted_items):
    # input already sorted by weight: O(n) with two queues
```

```
    from collections import deque
    leaves = deque((w, [s]) for s, w in sorted_items)
    merged = deque()
    depth = {s: 0 for s, _ in sorted_items}

    def pop_smaller():
        # the smaller front of the two queues
        if not merged or (leaves and
                          leaves[0][0] <= merged[0][0]):
            return leaves.popleft()
        return merged.popleft()

    while len(leaves) + len(merged) > 1:
        w1, a = pop_smaller()
        w2, b = pop_smaller()
        for s in a + b:
            depth[s] += 1
```

```
        merged.append((w1 + w2, a + b))
    return depth


# HELPERS for the checks below
def decode(bits, codes):
    # read bits until they match a code (prefix-free)
    back = {v: k for k, v in codes.items()}
    out, cur = [], ""
    for b in bits:
        cur += b
        if cur in back:
            out.append(back[cur])
            cur = ""
    return "".join(out)


if __name__ == "__main__":
    freq = {"a": 5, "b": 9, "c": 12,
            "d": 13, "e": 16, "f": 45}
    lengths = huffman_lengths(freq)
    # the most frequent symbol is next to the root
    assert lengths["f"] == 1
    # total cost of the optimal tree
    assert sum(freq[s] * lengths[s] for s in freq) == 224
    codes = huffman_codes(freq)
    # prefix-free: no code starts another code
    for x in codes.values():
        for y in codes.values():
            assert x == y or not y.startswith(x)
    text = "abfadc"
    bits = "".join(codes[ch] for ch in text)
    assert decode(bits, codes) == text
    items = sorted(freq.items(), key=lambda x: x[1])
    two = lengths_two_queues(items)
    assert sum(freq[s] * two[s] for s in freq) == 224
    assert huffman_codes({"x": 3}) == {"x": "0"}
    assert huffman_codes({}) == {}
```

NOTES FROM CANDIDATES

L6 onsite — interviewer expects clean code with a documented tree node, plus a discussion of encoding / decoding correctness.
Common follow-up: how to produce the code table (DFS from root, accumulating bits).
Trade-off discussion: Huffman is optimal for known per-symbol probability but not for adaptive / streaming inputs.

PREPARATION

Write the heap-based Huffman build from memory in under 15 min.
Drill the encode + decode helpers (DFS to build the code map; tree walk to decode).
Be ready to compare against arithmetic coding (better compression, costlier).

STUDY HERE: topic 5, Trees and Tries | main topic: Tries

Also fits: Graphs and Search, Strings, Parsing and Recursion, Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: string, parsing, data-structure, trie, dfs

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 8 of 10

URL Request Handler Router

ASKED 2026-10-16 | 4 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 222/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Tries

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/url-request-handler-router

QUESTION

EXAMPLES

```
Register /users/{userId}/photo/{photoId}. A lookup
    for /users/123/photo/456 matches that registered
    handler.
```

APPROACH

In 15 minutes, implement the scan baseline and check the existing example, segment-count mismatches, and no-match cases under an explicitly chosen return convention.
In 25 minutes, build the segment trie and compare its complete match set with the scan on small generated inputs. Include a literal branch that fails later and patterns with different placeholder names but the same structure.
In 10 minutes, write a contract checklist for precedence, duplicate registration, trailing slashes, query strings, and URL decoding; separate agreed rules from assumptions in your tests.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how to choose between several matching patterns (literal over placeholder?), what to return on no match or duplicate registration, and how to treat trailing slashes, query strings and case.

BRUTE FORCE TO SAY OUT LOUD

Store the patterns in a list and compare segment by segment. O(H * L) per lookup. It is a correct baseline and gives a reference to test the trie against.

OPTIMAL IDEA

Segment trie: each node has literal children and one wildcard child, with the original pattern stored at the end node. Lookup explores the literal child first and the wildcard child on failure, because a literal branch can die later. Accept only when all segments are consumed and a handler is stored.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

An exact route, a placeholder route, both literal and placeholder matching the same URL, segment-count mismatch, no match.

LIKELY FOLLOW-UPS

Methods (GET and POST), wildcard at the end, priority rules, and many routes.

PITFALLS

Picking the literal branch greedily without falling back. Overwriting patterns that differ only in placeholder names.

HOW TO THINK ABOUT IT

Matching a path against many patterns is a prefix problem, so share the prefixes in a trie of segments. The catch is the wildcard: a literal branch can fail later, so you cannot pick it greedily. Try the literal child first and fall back to the wildcard child. That also encodes the precedence rule for free.

Basis: Written. Assumed rules (confirm them): patterns look like /users/{id}/photo; {name} matches exactly one segment; a literal beats a placeholder at the first segment where they differ; no match returns None; trailing slashes are ignored.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def lookup(url):
    parts = url.split("/")
    matches = []
    for pattern in patterns:
        p = pattern.split("/")
        if len(p) != len(parts):
            continue
        if all(is_param(x) or x == y
               for x, y in zip(p, parts)):
            matches.append(pattern)
    return best_by_precedence(matches)
```

TIME	SPACE
O(H * L) per lookup	**O(1)** extra

WHY IT IS SLOW

Each lookup compares against every registered pattern, O(patterns * segments). A trie of segments with a literal child and a wildcard child follows only the compatible branches.

OPTIMAL SOLUTION · Segment trie with literal-first fallback

TIME	SPACE
O(L * branching) per lookup, O(total pattern length) to build	**O(total pattern length)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Different HTTP methods per route.

Make the method the first segment of the path (a tiny trick) or keep a dict method to trie. Lookup uses the method as the first step.

FOLLOW-UP 2 A wildcard at the end that matches the rest of the path (/static/*).

Give each trie node a 'rest' handler. During lookup, remember the deepest node that has one and use it if the exact walk fails. Precedence: exact, then placeholder, then rest-wildcard.

FOLLOW-UP 3 Return the extracted parameters (id = 7).

Carry a list of (name, segment) while walking; append when you take a wildcard edge and drop it when you backtrack. Store the placeholder names with the pattern, not in the trie.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: URL Request Handler Router
#
# Examples: Register /users/{userId}/photo/{photoId}. A
# lookup for /users/123/photo/456 matches that registered
# handler.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Different HTTP methods per route.
# 2. A wildcard at the end that matches the rest of the
# path (/static/*).
# 3. Return the extracted parameters (id = 7).
#
# INTERVIEW NOTES
# Matching a path against many patterns is a prefix
# problem, so share the prefixes in a trie of segments.
# The catch is the wildcard: a literal branch can fail
# later, so you cannot pick it greedily. Try the literal
# child first and fall back to the wildcard child. That
# also encodes the precedence rule for free.
#
# BRUTE FORCE (say it first): Scan every registered
# pattern
# Compare the URL's segments with each pattern's segments
# (same count, each one equal or a placeholder), collect
# the matches and keep the best by specificity. It is the
# correctness baseline, and a good oracle for the trie.
# Time O(H * L) per lookup. Space O(1) extra.


# HELPERS for the brute force
def segments(path):
    # "/users/7/photo" -> ["users", "7", "photo"]
    return [s for s in path.split("/") if s]


# BRUTE FORCE (say it first): try every route in a list
def fits(pat, segs):
    # the placeholder values if the route fits, else None
    params = {}
    for i, p in enumerate(pat):
        if p == "*":
            return params if len(segs) > i else None
        if i >= len(segs):
            return None
        if p.startswith("{") and p.endswith("}"):
            params[p[1:-1]] = segs[i]
        elif p != segs[i]:
            return None
    return params if len(pat) == len(segs) else None


class RouterBrute:
    def __init__(self):
        self.routes = []

    def add(self, pattern, handler=None, method="GET"):
        pat = [method] + segments(pattern)
        self.routes.append((pat, handler or pattern))

    def match(self, url, method="GET"):
        segs = [method] + segments(url)
        best = None
        for pat, handler in self.routes:
            params = fits(pat, segs)
            if params is None:
                continue
            # literal beats placeholder beats trailing *
            score = [2 if p == s else 1 if p[0] == "{"
                     else 0 for p, s in zip(pat, segs)]
            if best is None or score >= best[0]:
                best = (score, handler, params)
        return (best[1], best[2]) if best else (None, {})


# HELPERS for the base question
class Node:
    def __init__(self):
        # fixed segment -> child node
        self.literal = {}
        # child for a {placeholder} segment
        self.wild = None
        self.wild_name = None
        # handler when the path ends here
        self.handler = None
        # handler for a trailing "*" (rest of the path)
        self.rest = None


# BASE QUESTION (optimal): Segment trie with literal-first
# fallback
```

```
# Idea: Each trie node has literal children and one
# wildcard child; the pattern is stored at its last node.
# Lookup tries the literal child first, and if that whole
# branch fails it backs up and tries the wildcard child.
# FOLLOW-UP 1: Different HTTP methods per route.
# Answer: Make the method the first segment of the path (a
# tiny trick) or keep a dict method to trie. Lookup uses
# the method as the first step.
class Router:
    def __init__(self):
        self.root = Node()

    def add(self, pattern, handler=None, method="GET"):
        node = self.root
        # FOLLOW-UP 1: the HTTP method is the first
        # segment
        # the method is simply the first segment
        for seg in [method] + segments(pattern):
            # FOLLOW-UP 2: a trailing * matches the rest
            # of the path
            if seg == "*":
                # all deeper paths use this handler
                node.rest = handler or pattern
```

```
                return
            if seg.startswith("{") and seg.endswith("}"):
                node.wild = node.wild or Node()
                node.wild_name = seg[1:-1]
                node = node.wild
            else:
                lit = node.literal
                node = lit.setdefault(seg, Node())
        node.handler = handler or pattern

    def match(self, url, method="GET"):
        segs = [method] + segments(url)
        params = {}

        def walk(node, i):
            if i == len(segs):
                return node.handler
            # 1) exact segment first
            child = node.literal.get(segs[i])
            if child is not None:
                found = walk(child, i + 1)
                if found is not None:
```

```
                    return found
            # 2) then a placeholder, remembering its value
            if node.wild is not None:
                # FOLLOW-UP 3: remember the value (id = 7)
                params[node.wild_name] = segs[i]
                found = walk(node.wild, i + 1)
                if found is not None:
                    return found
                # undo the value when backtracking
                params.pop(node.wild_name, None)
            # 3) finally a trailing wildcard
            return node.rest

        handler = walk(self.root, 0)
        return handler, dict(params)


if __name__ == "__main__":
    r = Router()
    r.add("/users/{userId}/photo/{photoId}", "photo")
    r.add("/users/me", "me")
    r.add("/static/*", "static")
    r.add("/orders/{id}", "create", method="POST")
    hit = r.match("/users/123/photo/456")
    ids = {"userId": "123", "photoId": "456"}
    assert hit == ("photo", ids)
    # a literal beats a placeholder
    assert r.match("/users/me")[0] == "me"
    assert r.match("/users/me/photo/9")[0] == "photo"
    # trailing wildcard takes the rest
    assert r.match("/static/css/app.css")[0] == "static"
    # different method, different route
    post = r.match("/orders/5", "POST")
    assert post == ("create", {"id": "5"})
    assert r.match("/orders/5")[0] is None
    # the brute force agrees with the trie
    import random
    words = ["a", "b", "{}", "*"]
    for _ in range(300):
        fast, slow = Router(), RouterBrute()
        for k in range(random.randint(1, 5)):
            n = random.randint(1, 3)
            parts = [random.choice(words)
                     for _ in range(n)]
            if "*" in parts:
```

```
                parts = parts[:parts.index("*") + 1]
            # a placeholder is named after its position
            parts = ["{p%d}" % i if p == "{}" else p
                     for i, p in enumerate(parts)]
            pattern = "/" + "/".join(parts)
            fast.add(pattern, f"h{k}")
            slow.add(pattern, f"h{k}")
        for _ in range(8):
            n = random.randint(1, 3)
            url = "/" + "/".join(random.choice("abc")
                                 for _ in range(n))
            assert fast.match(url) == slow.match(url), url
```

NOTES FROM CANDIDATES

Clarify how to choose between multiple matching handlers, including a literal segment competing with a placeholder. The precedence rule is unspecified.

The no-match result, duplicate-registration behavior, and URL normalization rules are also unspecified; do not assume a canonical behavior before discussing them.

Matching approach

Start with a scan of registered patterns: compare segment counts, then require each position to contain either the same literal or a placeholder. This is a correctness baseline that can collect all matches before applying the agreed precedence rule.

A segment trie shares prefixes. Keep literal children and a separate wildcard child; at each URL segment explore both compatible branches. A literal branch can fail later, so choosing it greedily can miss a valid wildcard match.

Store the original registered patterns at terminal nodes. Patterns with different placeholder names can share the same trie path; retain each original rather than overwriting it. Accept terminals only after consuming the entire URL.

For tokenized paths of L segments and H handlers, the scan costs O(HL) segment comparisons in the worst case. A wildcard trie can branch, so lookup is not necessarily O(L): account for visited states, matching candidates, and string-processing costs separately. Keep normalization

and winner selection tied to the clarified contract.

PREPARATION

In 15 minutes, implement the scan baseline and check the existing example, segment-count mismatches, and no-match cases under an explicitly chosen return convention.

In 25 minutes, build the segment trie and compare its complete match set with the scan on small generated inputs. Include a literal branch that fails later and patterns with different placeholder names but the same structure.

In 10 minutes, write a contract checklist for precedence, duplicate registration, trailing slashes, query strings, and URL decoding; separate agreed rules from assumptions in your tests.

STUDY HERE: topic 5, Trees and Tries | main topic: Tries

Also fits: Graphs and Search (listed there as SKIP, no pages). Site tags: trie, tree, dfs, medium

MEDIUM | IN APP LC 208 | 1Point3Acres | tech-screen, onsite-coding | 9 of 10

Trie Prefix Search / Autocomplete

ASKED 2026-07-02 | 110 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 36** **30 DAYS 0** **PRIORITY 108/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Tries

Optimal answer from: your app's interview approach (LeetCode #208)

LeetCode match: LC 208 Implement Trie (Prefix Tree)

Frequency: Medium | Last asked: 2026-07-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/trie-prefix-search-autocomplete

QUESTION

LeetCode 208 + 1268. Build a trie from a dictionary and return all words sharing a query prefix. Standard Google onsite coding.

Build a trie from a dictionary / array of words.
Support searchPrefix(prefix) -> list[str]: return all words that start with the query prefix, lexicographically sorted unless ranking is specified.
Some interviewers phrase the query as "a few letters" or "substring"; clarify whether they truly mean arbitrary substring search or the standard prefix/autocomplete contract before coding.
Common follow-ups: deletion, fuzzy match (one-character edit), autocompletion ranking by frequency, and returning only the best K completions.

EXAMPLES

```
Dict ["apple","app","apricot","banana"], prefix "ap"
    -> ["app","apple","apricot"].
```

APPROACH

Write the trie class from memory in under 12 min.
Drill the DFS-collect and the sorted-per-node variants; know which is faster in which scenario.
For frequency-ranking, practice the design-autocomplete variant (per-node top-K updated on insert) as a stretch goal.

HOW TO THINK ABOUT IT

A trie shares prefixes, so walking down the query prefix costs O(length of the prefix) no matter how many words exist. Everything under that node is a completion. For autocomplete you either collect the words below the prefix node with a DFS (visiting children in sorted order gives sorted output) or store the best few words at every node while inserting.

Basis: LeetCode 208, Implement Trie, plus LeetCode 1268, Search Suggestions System (top completions per prefix).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def search_prefix(words, prefix):
    found = []
    for w in words:
        if w.startswith(prefix):
            found.append(w)
    return sorted(found)
```

TIME	SPACE
O(n * L + n log n) per query	**O(output)**

WHY IT IS SLOW

Every query reads the whole dictionary. A trie reaches the prefix in O(length of the prefix) and reads only the completions.

OPTIMAL SOLUTION · Trie with a DFS over sorted children

TIME	SPACE
O(L + output) per query after O(total characters) to build	**O(total characters)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return only the best K completions ranked by frequency.

Store at every node the K best (frequency, word) pairs and update them while inserting; a query then reads one node in O(L + K) and never walks the subtree. The price is more memory and slower inserts, which is the right trade when reads dominate (autocomplete).

FOLLOW-UP 2 Allow one wrong character (fuzzy match).

Walk the trie with a budget of one edit: at each level follow the matching child with the full budget and every other child with the budget reduced to 0. The work grows with the alphabet size times the prefix length, not with the dictionary size.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Trie Prefix Search / Autocomplete
#
# LeetCode 208 + 1268. Build a trie from a dictionary and
# return all words sharing a query prefix. Standard Google
# onsite coding. Build a trie from a dictionary / array of
# words. Support searchPrefix(prefix) -> list[str]: return
# all words that start with the query prefix,
```

```
# lexicographically sorted unless ranking is specified.
# Some interviewers phrase the query as "a few letters" or
# "substring"; clarify whether they truly mean arbitrary
# substring search or the standard prefix/autocomplete
# contract before coding. Common follow-ups: deletion,
# fuzzy match (one-character edit), autocompletion ranking
# by frequency, and returning only the best K completions.
# Examples: Dict ["apple","app","apricot","banana"],
# prefix "ap" -> ["app","apple","apricot"].
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only the best K completions ranked by
# frequency.
# 2. Allow one wrong character (fuzzy match).
#
# INTERVIEW NOTES
# A trie shares prefixes, so walking down the query prefix
# costs O(length of the prefix) no matter how many words
# exist. Everything under that node is a completion. For
# autocomplete you either collect the words below the
# prefix node with a DFS (visiting children in sorted
# order gives sorted output) or store the best few words
# at every node while inserting.
#
# BRUTE FORCE (say it first): Filter the whole word list
# Scan every word, keep those starting with the prefix and
# sort them. Correct and the oracle; every query pays for
# the whole dictionary. Time O(n * L + n log n) per query.
# Space O(output).


# BRUTE FORCE (say it first): Filter the whole word list |
# time O(n * L + n log n) per query
def prefix_words_brute(words, prefix):
    return sorted(
        {w for w in words if w.startswith(prefix)}
    )
```

```
# HELPERS for the base question
class Node:
    def __init__(self):
        self.children = {}
        # how often the word ending here was inserted
        self.freq = 0
        # best (negative freq, word) pairs under this node
        self.top = []


# BASE QUESTION (optimal): Trie with a DFS over sorted
# children
# Idea: Insert each word, marking the last node. For a
# query walk the prefix; if a step is missing return an
# empty list.
class Trie:
    def __init__(self, k=3):
        self.root, self.k = Node(), k

    def insert(self, word, freq=1):
        node = self.root
        path = [node]
        for ch in word:
            node = node.children.setdefault(ch, Node())
            path.append(node)
        node.freq += freq
        # refresh the top-k list on every node of the path
        for n in path:
            n.top = [x for x in n.top if x[1] != word]
            n.top.append((-node.freq, word))
            n.top.sort()
            del n.top[self.k:]

    def find(self, prefix):
        # walk down the prefix; None if it is missing
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def search_prefix(self, prefix):
        # every word with this prefix, in alphabetical
        # order
        node = self.find(prefix)
        out = []
        if node is None:
            return out
        stack = [(node, prefix)]
        while stack:
            n, text = stack.pop()
            if n.freq:
                out.append(text)
            for ch in n.children:
                stack.append((n.children[ch], text + ch))
        return sorted(out)

    def suggest(self, prefix):
        # best k completions, read from one node: O(L + k)
        node = self.find(prefix)
        return [w for _, w in node.top] if node else []

    def fuzzy(self, prefix):
        # words whose first letters differ in at most one
        # place
        out, stack = [], [(self.root, "", 0)]
        while stack:
            n, text, wrong = stack.pop()
            if len(text) == len(prefix):
                out += self.search_below(n, text)
                continue
            for ch, child in n.children.items():
                miss = wrong + (ch != prefix[len(text)])
                if miss <= 1:
                    stack.append((child, text + ch, miss))
        return sorted(out)

    def search_below(self, node, text):
        out, stack = [], [(node, text)]
        while stack:
            n, t = stack.pop()
            if n.freq:
                out.append(t)
            for ch, child in n.children.items():
                stack.append((child, t + ch))
        return out


if __name__ == "__main__":
    t = Trie(k=2)
    for w in ["apple", "app", "apricot", "banana"]:
        t.insert(w)
    ap = t.search_prefix("ap")
    assert ap == ["app", "apple", "apricot"]
    assert t.search_prefix("x") == []
    assert t.search_prefix("") == [
        "app", "apple", "apricot", "banana"]
    # frequency ranking: app was typed three more times
    t.insert("app", 3)
    assert t.suggest("ap") == ["app", "apple"]
    assert t.suggest("zzz") == []
    # one wrong letter in the prefix is allowed
    assert "banana" in t.fuzzy("vana")
    assert t.fuzzy("zzzz") == []
```

NOTES FROM CANDIDATES

Standard implementation: each trie node has a children map and an is_end flag.
Two collection patterns at the prefix: (a) DFS from the prefix node and collect all word-end nodes, (b) store the top-K candidates at every node during insert (Google-preferred for autocomplete).
The standard sorted-suggestions follow-up wants 3 suggestions per prefix-length — handle with sorted children or per-node sorted lists.

PREPARATION

Write the trie class from memory in under 12 min.
Drill the DFS-collect and the sorted-per-node variants; know which is faster in which scenario.
For frequency-ranking, practice the design-autocomplete variant (per-node top-K updated on insert) as a stretch goal.

STUDY HERE: topic 5, Trees and Tries | main topic: Tries

Also fits: Arrays, Hashing and Two Pointers, Strings, Parsing and Recursion, Greedy and Math (listed there as SKIP, no pages). Site tags: string, trie, hashmap, greedy, medium

MEDIUM | 1Point3Acres | phone-screen | 10 of 10

Greedy Longest-Match Dictionary Tokenizer

ASKED 2026-06-15 | 127 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 215/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Tries

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-06-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/tokenize-longest-dict-match-greedy

QUESTION

Phone-screen coding round (R1). Tokenize a text against a dictionary of `key:id` entries using greedy longest-match; emit the matched id when a key matches, otherwise output the unmatched character as a literal. One follow-up on changing the tie-breaking / priority rule for which token wins. A second follow-up appears when the dictionary is very large: discuss how to keep the matcher memory- and lookup-efficient at scale.

Input: a text string and a dictionary mapping key (string) -> id.
Sweep the text left-to-right. At each position, find the longest dictionary key that matches a prefix of the remaining text; emit its id. If no key matches at the current position, emit the single character as a literal and advance by one.
Output the concatenation (or list) of ids and literal characters in order.
Follow-up: the interviewer changes the tie-breaking / priority rule for which key wins when multiple match (e.g., not longest but highest-priority by dictionary order). Adapt the data structure so the rule swap is cheap.
Follow-up: the dictionary becomes very large. Discuss how to keep memory and per-position lookup efficient — a compressed / radix trie (or sharding the key space) instead of holding every key in a flat map.

APPROACH

Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.
For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.
If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.

HOW TO THINK ABOUT IT

At each position you want the longest dictionary key that is a prefix of the remaining text. Putting the keys in a trie lets you walk the text character by character from that position and remember the deepest end-of-key node you passed, instead of trying every length. If the tie-break rule changes (priority instead of length), store the priority at the terminal node and keep the best terminal seen.

Basis: Written for this question (longest-prefix matching with a trie, as in LeetCode 208 and the maximal-munch rule of lexers).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def tokenize(text):
    out, i = [], 0
    while i < len(text):
        for size in range(max_key_len, 0, -1):
            key = text[i:i + size]
            if key in dictionary:
                out.append(dictionary[key])
                i += size
                break
        else:
            out.append(text[i])  # literal
            i += 1
    return out
```

TIME	SPACE
O(n * L) lookups, each O(L)	O(dictionary)

WHY IT IS SLOW

Every lookup hashes a substring, O(L^2) per position. Walking a trie of the keys one character at a time costs O(L) per position.

OPTIMAL SOLUTION · Trie walk keeping the deepest (or best-priority) terminal

TIME	SPACE
O(n * L)	O(total key length)

OPTIMAL SOLUTION · Trie walk keeping the deepest (or best-priority) terminal

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The dictionary is huge.

A flat map of every key wastes memory because keys share prefixes. A compressed (radix) trie stores each shared stretch once, or shard the keys by first characters across machines. Lookup stays one pass over the text; only the node layout changes.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Greedy Longest-Match Dictionary Tokenizer
#
# Phone-screen coding round (R1). Tokenize a text against
# a dictionary of key:id entries using greedy longest-
# match; emit the matched id when a key matches, otherwise
# output the unmatched character as a literal. One follow-
# up on changing the tie-breaking / priority rule for
# which token wins. A second follow-up appears when the
# dictionary is very large: discuss how to keep the
# matcher memory- and lookup-efficient at scale. Input: a
# text string and a dictionary mapping key (string) -> id.
# Sweep the text left-to-right. At each position, find the
# longest dictionary key that matches a prefix of the
# remaining text; emit its id. If no key matches at the
# current position, emit the single character as a literal
# and advance by one. Output the concatenation (or list)
# of ids and literal characters in order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The dictionary is huge.
#
# INTERVIEW NOTES
# At each position you want the longest dictionary key
# that is a prefix of the remaining text. Putting the keys
# in a trie lets you walk the text character by character
# from that position and remember the deepest end-of-key
# node you passed, instead of trying every length. If the
# tie-break rule changes (priority instead of length),
# store the priority at the terminal node and keep the
# best terminal seen.
#
# BRUTE FORCE (say it first): Hash map lookup for every
# prefix length
# At every position try the prefix lengths from longest to
# shortest in a dictionary lookup. Correct and easy; each
# lookup hashes a substring, so it costs O(L^2) per
# position. Time O(n * L) lookups, each O(L). Space
# O(dictionary).


# BRUTE FORCE (say it first): Hash map lookup for every
# prefix length | time O(n * L) lookups, each O(L)
def tokenize_brute(text, table):
    out, i, longest = (
        [],
```

```
        0,
        max(map(len, table), default=0),
    )
    while i < len(text):
        for k in range(
            min(longest, len(text) - i), 0, -1
        ):
            if text[i : i + k] in table:
                out.append(table[text[i : i + k]])
                i += k
                break
        else:
            out.append(text[i])
            i += 1
    return out


# BASE QUESTION (optimal): Trie walk keeping the deepest
# (or best-priority) terminal
# Idea: Insert every key into a trie with its id at the
# last node. From each text position walk down while the
# next character exists, remembering the last terminal
# passed.
# FOLLOW-UP 1: The dictionary is huge.
# Answer: A flat map of every key wastes memory because
# keys share prefixes. A compressed (radix) trie stores
# each shared stretch once, or shard the keys by first
# characters across machines.
class Tokenizer:
    def __init__(self, table, priority=None):
        # table: key -> id. priority(key): lower wins;
        # with
        # None the longest match wins
        self.priority = priority
        self.root = {}
        for key, ident in table.items():
            node = self.root
            for ch in key:
                node = node.setdefault(ch, {})
            # the end of a key stores the key and its id
            node["$"] = (key, ident)

    def better(self, new, old):
        # should the match `new` replace the match `old`?
        if old is None:
```

```
            return True
        if self.priority is None:
            # deeper (longer) matches come later: keep
            # them
            return True
        rank = self.priority
        return rank(new[0]) < rank(old[0])

    def longest_at(self, text, i):
        # walk the trie from position i
        node, best, j = self.root, None, i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            end = node.get("$")
            if end and self.better(end, best):
                best = end
        return best

    def tokenize(self, text):
        out, i = [], 0
        while i < len(text):
            match = self.longest_at(text, i)
            if match is None:
                # no key starts here: keep the character
                out.append(text[i])
                i += 1
            else:
                key, ident = match
                out.append(ident)
```

```
                i += len(key)
        return out


# SIMPLER VERSION: one dictionary lookup per length
def tokenize_map(text, table):
    # simpler version: a dictionary lookup per length
    out, i = [], 0
    longest = max(map(len, table), default=0)
    while i < len(text):
        top = min(longest, len(text) - i)
        for size in range(top, 0, -1):
            if text[i:i + size] in table:
                out.append(table[text[i:i + size]])
                i += size
```

```
                break
        else:
            out.append(text[i])
            i += 1
    return out


if __name__ == "__main__":
    table = {"ab": 1, "abc": 2, "c": 3}
    t = Tokenizer(table)
    # abc is longer than ab, so it wins at position 0
    assert t.tokenize("abcab") == [2, 1]
    assert t.tokenize("xyz") == ["x", "y", "z"]
    assert t.tokenize("") == []
    assert tokenize_map("abcab", table) == [2, 1]
    # a single-letter key still applies the longest rule
    assert t.tokenize("cab") == [3, 1]
    # follow-up: priority by dictionary order instead
    order = list(table)
    pri = Tokenizer(table, priority=order.index)
    assert pri.tokenize("abc") == [1, 3]
```

NOTES FROM CANDIDATES

The key:id dictionary format and the literal-passthrough for unmatched characters are confirmed across multiple candidates.
Two natural shapes:

Hashmap of keys + per-position scan up to maxKeyLen. Easy to write, O(n · maxKeyLen) time; O(n · L) if the longest key length L dominates.
Trie of keys + walk the trie one character at a time from each position; remember the deepest reached is_end node and emit it. O(n · L) worst case but does only one pass per starting position and adapts cleanly to the priority-rule follow-up (store priority at each terminal node and keep the best matched terminal along the walk, not necessarily the deepest).

Edge cases to discuss: overlapping keys with different lengths ("ab" vs "abc"), key whose match would consume the rest of the string, empty dictionary, key that is a single character (still apply longest-match, not literal-fallback).
If the follow-up generalizes to "non-overlapping tokenization that maximizes total score", that's the word-break with weights DP variant — flag it as a different problem class rather than patching the greedy.

PREPARATION

Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.
For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.
If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.